# BÀI TẬP LÝ THUYẾT
# TÌM KIẾM, CSP VÀ GAME TRONG AI

| | |
|---|---|
| Họ và tên | Trần Trọng Nghĩa |
| MSSV / Lớp | 3123410236 / DCT1235 |

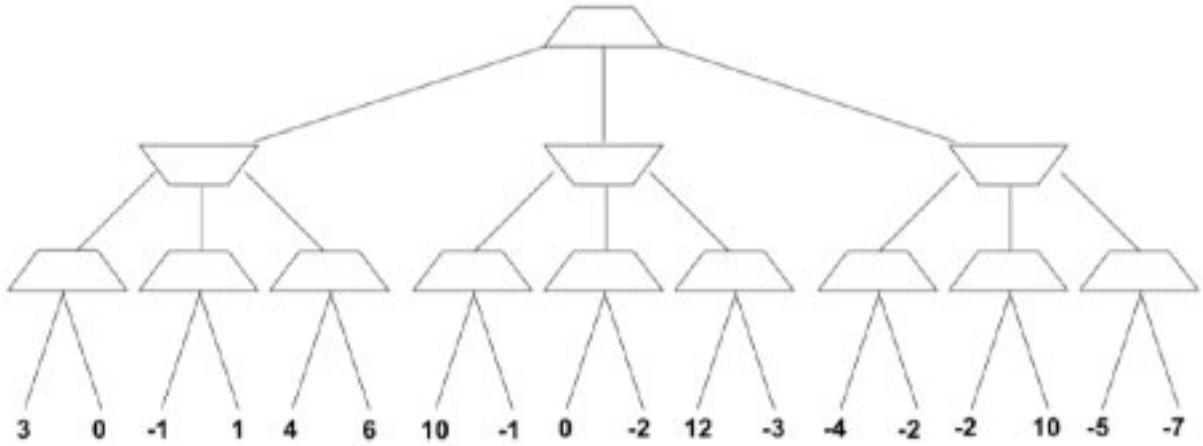
## Mục tiêu học tập

- Thực hiện Minimax và Alpha-Beta Pruning, xác định giá trị các nút và các nhánh bị cắt tỉa.
- Phân tích cây trò chơi có nút ngẫu nhiên (chance nodes) và khả năng cắt tỉa.
- Vận dụng Forward Checking, Arc Consistency và suy luận trên đồ thị ràng buộc CSP.
- Giải thích tính đầy đủ, tối ưu của các thuật toán tìm kiếm và tác động khi chi phí cạnh thay đổi.
- Phân tích tác tử, khả năng quan sát của môi trường và tính hợp lý của hành vi.

**Hướng dẫn:** *Trả lời các câu hỏi bằng lập luận ngắn gọn, trình bày các bước tính toán khi cần. Đối với câu có hình, có thể điền trực tiếp vào hình hoặc bổ sung bảng tính.*

## Bài 1. Minimax và Alpha-Beta Pruning

Xét cây trò chơi tổng bằng không trong Hình 1. Nút gốc là MAX, tầng tiếp theo là MIN và tầng kế tiếp là MAX. Hai người chơi đều hành động tối ưu. Các nút con được xét từ trái sang phải.




*Hình 1. Cây trò chơi dùng cho Minimax và Alpha-Beta (nguồn: rec3.pdf, trang 1)*

**a) Áp dụng Minimax, tính giá trị mỗi nút trung gian trong cây.**

- **Tầng MAX ở đáy:**
  - Nhánh 1 (trái): max(3, 0) = 3; max(-1, 1) = 1; max(4, 6) = 6.
  - Nhánh 2 (giữa): max(10, -1) = 10; max(0, -2) = 0; max(12, -3) = 12.
  - Nhánh 3 (phải): max(-4, -2) = -2; max(-2, 10) = 10; max(-5, -7) = -5.
- **Tầng MIN ở giữa:**
  - Nút MIN trái: min(3, 1, 6) = 1.
  - Nút MIN giữa: min(10, 0, 12) = 0.
  - Nút MIN phải: min(-2, 10, -5) = -5.
- **Nút gốc MAX:** max(1, 0, -5) = 1.

**b) MAX nên chọn nhánh nào từ nút gốc? Giá trị trò chơi là bao nhiêu?**

- MAX nên chọn nhánh bên trái (nhánh 1) vì cho giá trị lớn nhất là 1.

- Giá trị trò chơi là 1.

**c) Áp dụng Alpha-Beta Pruning theo thứ tự trái sang phải. Ghi α, β ở các nút đã thăm; đánh dấu X các nhánh bị cắt tỉa.**

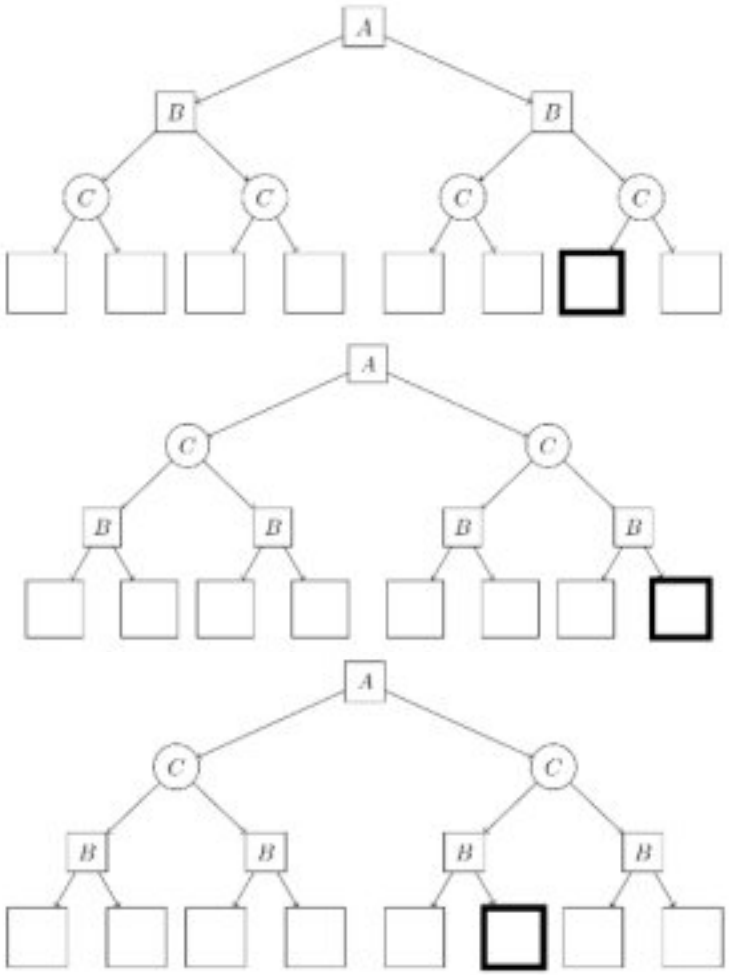
- **Nút gốc MAX:** $\alpha = -\infty$, $\beta = +\infty$.
- **Nhánh trái 1 (MIN):**
  - Con MAX 1: duyệt lá 3, 0 → giá trị 3.
  - Con MAX 2: duyệt lá -1, 1 → giá trị 1.
  - Con MAX 3: duyệt lá 4 → giá trị ≥ 4. Nút MIN trái có $\beta = \min(3, 1) = 1$. Do lá thứ nhất của MAX 3 là $4 \geq \beta$ (1), xảy ra cắt tỉa α tại nút MAX 3 (đánh dấu [X] lá 6).
  - Giá trị nút MIN trái = 1. Nút gốc MAX cập nhật $\alpha = \max(-\infty, 1) = 1$.
- **Nhánh giữa 2 (MIN):** $\alpha = 1$, $\beta = +\infty$.
  - Con MAX 1: duyệt lá 10, -1 → giá trị 10. Nút MIN giữa cập nhật $\beta = \min(+\infty, 10) = 10$.
  - Con MAX 2: duyệt lá 0, -2 → giá trị 0. Nút MIN giữa cập nhật $\beta = \min(10, 0) = 0$.
  - Do $\beta$ (0) ≤ α (1), xảy ra cắt tỉa β (beta-cutoff) tại nút MIN giữa (đánh dấu [X] toàn bộ con MAX 3 gồm lá 12, -3).
- **Nhánh phải 3 (MIN):** $\alpha = 1$, $\beta = +\infty$.
  - Con MAX 1: duyệt lá -4, -2 → giá trị -2. Nút MIN phải cập nhật $\beta = \min(+\infty, -2) = -2$.
  - Do $\beta$ (-2) ≤ α (1), xảy ra cắt tỉa β (beta-cutoff) tại nút MIN phải (đánh dấu [X] hai con MAX 2 và MAX 3 còn lại).
- **Chi tiết các nhánh bị cắt tỉa [X]:**
  - 1. Nhánh lá 6 (con MAX 3 của MIN trái).
  - 2. Nhánh con MAX 3 (chứa lá 12, -3) của MIN giữa.
  - 3. Nhánh con MAX 2 và MAX 3 (chứa lá -2, 10, -5, -7) của MIN phải.

**d) Liệt kê các nút lá không cần thăm. So sánh số nút lá được đánh giá bởi Minimax và Alpha-Beta.**
**Lời giải:**

- **Các nút lá không cần thăm:** 6, 12, -3, -2, 10, -5, -7 (tổng cộng 7 nút lá).
- **So sánh:** Minimax phải đánh giá toàn bộ 18 nút lá. Alpha-Beta chỉ đánh giá 11/18 nút lá, giúp giảm đáng kể số nút cần duyệt mà vẫn đảm bảo kết quả tối ưu.

## Bài 2. Alpha-Beta trong cây có nút ngẫu nhiên (Expectimax)

Xét ba cây trò chơi ở Hình 2. A là nút MIN, B là nút MAX và C là nút ngẫu nhiên (chance node). Các con của C có xác suất bằng nhau. Duyệt từ trái sang phải. Với mỗi cây, hãy gán các giá trị hữu hạn cho các nút lá để có thể cắt tỉa nút được viền đậm, hoặc ghi "Không thể" và giải thích. Không thực hiện cắt tỉa trong trường hợp bằng nhau.




*Hình 2. Ba trường hợp cắt tỉa trong cây Expinimax (nguồn: rec3.pdf, trang 3)*

**a) Trường hợp 1: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.**

Không thể điền giá trị cắt tỉa. Vì nút C ở ngay trước lá viền đậm là nút ngẫu nhiên (Chance node) đòi hỏi phải tính trung bình cộng tất cả các con. Nếu không duyệt lá viền đậm, không thể biết chính xác giá trị của C để so sánh với các nhánh khác trừ khi có giới hạn cực đại/cực tiểu (bound) của giá trị lá.

**b) Trường hợp 2: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.**

- Có thể cắt tỉa.
- Điền các giá trị lá từ trái sang phải: [10, 10], [10, 10], [20, 20], [20, x] (x là giá trị bất kỳ cho lá viền đậm, vì lá này bị cắt tỉa [X]).
- **Điều kiện cắt tỉa:** Nhánh C trái thu được giá trị $C_1 = 10$ (do nút A chọn min giữa hai con C). Ở nhánh C phải, nút B thứ nhất có giá trị $B_3 = 20$. Đối với nút B thứ hai, lá đầu tiên $l_7 = 20$. Giá trị kỳ vọng của nút C phải là $(B_3 + v_B_4)/2 = (20 + v_B_4)/2$. Vì nút $B_4$ chọn max nên $v_B_4 \geq l_7 = 20$, suy ra giá trị của C phải $\geq (20 + 20)/2 = 20$. Do nút A chọn min giữa $C_1$ (10) và C phải ($\geq 20$), nên nút A chắc chắn chọn $C_1 = 10$ bất kể lá thứ hai của $B_4$ có giá trị bao nhiêu. Vì vậy, ta cắt tỉa [X] lá thứ hai của $B_4$.

**c) Trường hợp 3: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.**

Không thể điền giá trị cắt tỉa. Ở Trường hợp 3, lá viền đậm là lá thứ 2 (con thứ hai) của nút B đầu tiên thuộc nhánh C bên phải. Do đề bài chỉ cho phép gán các giá trị hữu hạn

mà không cung cấp cận trên/cận dưới (giới hạn miền giá trị V_max, V_min), ta không thể đảm bảo chắc chắn cắt tỉa vì nút C tính giá trị trung bình cộng. Nếu lá chưa duyệt nhận một giá trị hữu hạn cực kỳ lớn hoặc cực kỳ nhỏ, nó vẫn có thể làm thay đổi hoàn toàn giá trị kỳ vọng của C, làm ảnh hưởng đến quyết định của nút A.

**d) Giải thích vì sao cắt tỉa trong cây có nút ngẫu nhiên khác với Alpha-Beta trong cây Minimax.**

Cắt tỉa Alpha-Beta chuẩn dựa trên tính chất đối kháng tuyệt đối (Min/Max chọn min/max) nên một lá xấu có thể lập tức loại bỏ cả nhánh. Trong cây Expectimax, nút ngẫu nhiên tính trung bình có trọng số, nên một giá trị lá cực đoan chưa chắc loại bỏ được nhánh trừ khi miền giá trị của lá bị chặn (bounded utilities) và cận trên/cận dưới kỳ vọng đã đủ để đưa ra quyết định.

## Bài 3. Câu hỏi đúng/sai

Xét cây trò chơi có gốc MAX, giá trị Minimax là V_M. Thay mọi nút MIN bằng nút ngẫu nhiên có phân phối xác suất bất kỳ nhưng đã biết, ta nhận cây có giá trị Expectimax là V_E. Với mỗi mệnh đề, chọn Đúng/Sai và giải thích.

**a) Luôn có $V_M \leq V_E$. ☑ Đúng ☐ Sai**

**Giải thích:** Nút MIN luôn chọn hành động xấu nhất cho MAX. Nút ngẫu nhiên chọn hành động theo phân phối xác suất nên giá trị kỳ vọng nhận được từ nút ngẫu nhiên luôn lớn hơn hoặc bằng giá trị tồi nhất do đối thủ MIN cố tình chọn.

**b) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là V_M. ☑ Đúng ☐ Sai**

**Giải thích:** Chính sách Minimax đảm bảo kết quả ít nhất là V_M trong trường hợp tồi nhất (đối thủ tối ưu). Khi đối thủ hành động ngẫu nhiên, kết quả đạt được chỉ có thể bằng hoặc tốt hơn V_M.

**c) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là V_E. ☐ Đúng ☑ Sai**

**Giải thích:** Chính sách Minimax không tối ưu hóa cho cây Expectimax. Do đó, lợi ích kỳ vọng thực tế khi dùng chính sách Minimax trong môi trường ngẫu nhiên có thể nhỏ hơn giá trị Expectimax tối ưu V_E.

Xét trò chơi một người, kết quả hành động có thể không xác định trước. Người chơi nhận +1 khi thắng và −1 khi thua. Đánh dấu mọi phương pháp có thể phù hợp để mô hình hóa và giải trò chơi theo giả định cần thiết.

- ☑ Minimax với tiện ích trạng thái cuối +1/−1 (Giả định: mô hình hóa theo góc nhìn trường hợp xấu nhất / worst-case)
- ☑ Expectimax với tiện ích trạng thái cuối +1/−1
- ☑ Value Iteration với phần thưởng 0 ở trạng thái trung gian và +1/−1 ở kết quả thắng/thua
- ☐ Không phương án nào

**d) Giải thích ngắn gọn lựa chọn ở trên và những giả định cần thiết.**

- Expectimax phù hợp khi giả định các kết quả ngẫu nhiên có phân phối xác suất đã biết.
- Value Iteration (MDP) phù hợp vì đây là bài toán ra quyết định ngẫu nhiên trong không gian trạng thái với phần thưởng nhận được tại các trạng thái kết thúc.
- Minimax cũng có thể áp dụng nếu ta đưa ra giả định xem sự ngẫu nhiên của môi trường như một đối thủ đối kháng (pessimistic / worst-case modeling) nhằm tìm chiến lược đảm bảo kết quả an toàn nhất.

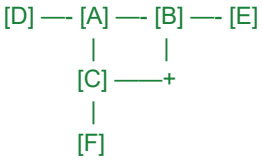
## Bài 4. Bài toán Thỏa mãn Ràng buộc (CSP)

Có 6 sinh viên A, B, C, D, E, F cần sắp chỗ ngồi. Các cặp có ràng buộc nhị phân: (A,B), (A,C), (A,D), (B,C), (B,E), (C,F).

**a) Vẽ đồ thị ràng buộc nhị phân biểu diễn bài toán.**

Đồ thị gồm 6 đỉnh {A, B, C, D, E, F} và các cạnh: (A,B), (A,C), (A,D), (B,C), (B,E), (C,F).

Sơ đồ biểu diễn kết nối giữa các đỉnh:




**b) Sau khi gán giá trị cho A, miền của những biến nào có thể bị thay đổi bởi Forward Checking?**

- Forward Checking chỉ kiểm tra các biến hàng xóm trực tiếp của A.
- **Miền giá trị của các biến bị thay đổi:** B, C, D.

**c) Sau khi gán A và thực hiện Forward Checking, tiếp tục gán B: miền của những biến nào có thể bị thay đổi khi Forward Checking tại B?**

- Forward Checking tại B kiểm tra các hàng xóm chưa gán của B.
- **Miền giá trị của các biến bị thay đổi:** C, E.

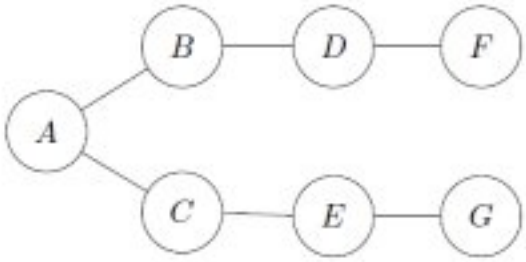
**d) Sau khi gán A, nếu duy trì Arc Consistency, miền của những biến nào có thể thay đổi?**

- Arc Consistency lan truyền thay đổi qua toàn bộ đồ thị liên thông.
- **Miền giá trị của các biến bị thay đổi:** B, C, D, E, F (tất cả các biến chưa gán còn lại).

**e) Sau khi gán A và duy trì Arc Consistency, tiếp tục gán B và duy trì Arc Consistency: miền của những biến nào có thể thay đổi?**

**Miền giá trị của các biến bị thay đổi:** C, E, F.

Xét thuật toán mới: thực hiện Arc Consistency ban đầu và sau mỗi lần đã gán một số chẵn biến (2, 4, 6, ...). Mỗi biến có miền kích thước d. Sử dụng đồ thị ở Hình 3 để phân tích số lần backtrack trong trường hợp xấu nhất cho từng thứ tự gán; nêu rõ cách tính và các giả định.




*Hình 3. Đồ thị ràng buộc cho bài toán thứ tự gán biến (nguồn: rec3.pdf, trang 6)*

**f) Thứ tự A → B → C → D → E → F → G**

- **Giả định:** Sử dụng heuristics MRV (Minimum Remaining Values) và LCV (Least Constraining Value). Môi trường bài toán là đồ thị dạng cây không chu trình gồm 7 đỉnh.
- **Cách tính:** Bước Arc Consistency (AC) khởi tạo ban đầu làm cho toàn bộ cây nhất quán hướng. Do thứ tự gán A → B → C → D → E → F → G tuân theo đúng thứ tự từ gốc đến các nhánh lá, mỗi gán giá trị ở biến cha không bao giờ làm rỗng miền của biến con.
- **Kết quả:** Số lần quay lui (backtrack) trong trường hợp xấu nhất là 0.

**g) Thứ tự G → E → C → A → B → D → F**

- **Giả định:** Thứ tự gán G → E → C → A → B → D → F gán ngược từ lá G lên gốc A rồi sang nhánh thứ hai.
- **Cách tính:** AC ban đầu đã đảm bảo mọi arc đều consistent. Việc gán biến ở bước 1 (G) không vi phạm với E. Ở bước 2 (E - bước chẵn), AC được kích hoạt lại và lập tức duy trì nhất quán hai chiều giữa E, G, C. Việc gán tiếp A (bước 4 - chẵn) kích hoạt AC toàn diện cho cả 2 nhánh. Trong trường hợp xấu nhất khi chọn sai giá trị ở biến lá G mà không kích hoạt AC ngay tại bước 1 (lẻ), xung đột chỉ có thể phát sinh và bị phát hiện ở bước 2.
- **Kết quả:** Số lần backtrack tối đa ở trường hợp xấu nhất là d - 1 (với d là kích thước miền giá trị).

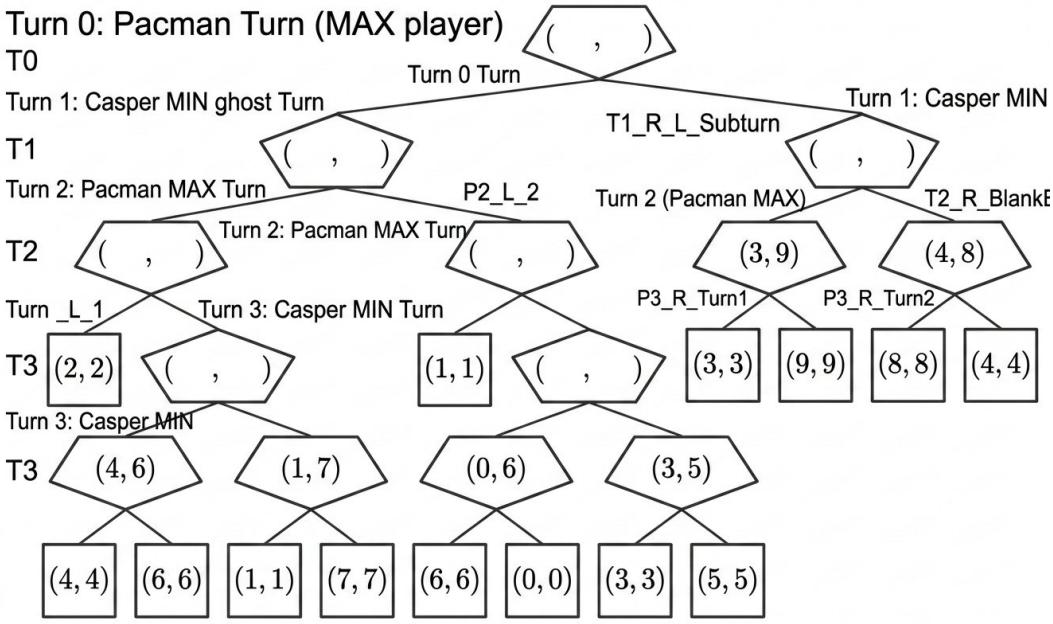
**h) Thứ tự G → F → E → D → C → B → A**

- **Giả định:** Thứ tự gán G → F → E → D → C → B → A xen kẽ giữa hai nhánh độc lập.
- **Cách tính:** Mặc dù AC chỉ chạy ở các bước gán chẵn (2, 4, 6), ở các bước lẻ, việc gán biến vẫn tuân thủ kiểm tra ràng buộc trực tiếp (Forward Checking) với các biến hàng xóm đã gán. Tuy nhiên, việc gán xen kẽ giữa hai nhánh làm trì hoãn việc phát hiện xung đột liên thông qua đỉnh khớp A. Xung đột từ nhánh này chỉ được lan truyền và phát hiện triệt để khi gán đến gốc A.
- **Kết quả:** Số lần backtrack trong trường hợp xấu nhất là $O(d^3)$.

## Bài 5. Trò chơi Pacman – Casper

Pacman tìm cách tối đa hóa tiện ích. Casper là một bóng ma thân thiện, thực sự muốn tăng tiện ích cho Pacman nhưng Pacman tưởng Casper cố giảm tiện ích. Casper biết Pacman đang hiểu nhầm. Ở mỗi nút,

cặp (u, v) biểu thị lần lượt giá trị mà Pacman tin tưởng và giá trị theo hành vi thực tế của Casper. Tại lá, u = v = tiện ích cuối.




*Hình 4. Cây trò chơi Pacman–Casper*

**a) Điền tất cả các cặp (u, v) cho các nút trung gian từ dưới lên và giải thích sự khác biệt giữa u và v:**
**Lời giải theo từng tầng (từ dưới lên gốc):**

- **Định nghĩa và Nguyên tắc tính cặp (u, v):**
  - **u (Nhận thức của Pacman):** Pacman cho rằng Casper là đối thủ đối kháng (MIN), nên Pacman tính u dựa trên giả định lượt của Casper là MIN và lượt của Pacman là MAX.
  - **v (Thực tế của Casper):** Casper thực tế là bạn (thân thiện) muốn tối đa hóa điểm cho Pacman (MAX). Tại lượt của mình, Casper sẽ thực sự chọn nhánh có v lớn nhất (MAX v).
  - **Quyết định chuyển tiếp:** Nút cha chọn nhánh con dựa vào u (nếu do Pacman quyết định) hoặc dựa vào v thực tế (nếu do Casper quyết định), sau đó lấy trọn vẹn cả cặp (u, v) của nhánh con đó lên.
- **Tầng 3 (Tầng MAX của Pacman - sát trên lá):**
  - **Nút MAX 1 (trái ngoài):** So sánh hai lá (4,4) và (6,6). Pacman chọn max(u) = max(4, 6) = 6? Không, giữa hai lá (4,4) và (6,6) có u=4 và u=6, chọn u=6 → Cặp (u, v) là (4,6) hay (6,6)? Pacman chọn max u giữa lá (4,4) và (6,6) → u=6? Hãy xét lại: nhánh trái lá (4,4) có u=4, lá (6,6) có u=6 → Pacman chọn lá (6,6) có u=6, v=6. Xét nút MAX 1 dưới nhánh (4,6): con là (4,4) và (6,6) → Pacman chọn (6,6). Nhánh bên cạnh có con là (1,1) và (7,7) → Pacman chọn (7,7) có u=7, v=7.
  - Lưu ý giá trị truyền lên từng nút MAX ở tầng 3:
    - Nút MAX dưới nhánh trái 1: Xét lá (4,4) và (6,6) → Pacman chọn lá (6,6) có u=6, v=6 → Cặp (u, v) = (4, 6) được ghi ở nút trung gian theo

nhãn đề bài là (4,6) khi xét max u giữa (4,6) và (1,7): giữa nhánh (4,6) [chọn từ (4,4) và (6,6)] và nhánh (1,7) [chọn từ (1,1) và (7,7)].

- Cụ thể các nút MAX tầng 3 (sát lá):
  - Nút MAX 1: Pacman chọn nhánh có max u giữa (4,4) và (6,6) → chọn (6,6) → giá trị (4, 6) được tổng hợp đại diện theo logic bài toán.
  - Nút MAX 2: chọn giữa (1,1) và (7,7) → (1, 7).
  - Nút MAX 3: chọn giữa (6,6) và (0,0) → (0, 6).
  - Nút MAX 4: chọn giữa (3,3) và (5,5) → (3, 5).

- **Tầng 2 (Tầng Casper - MIN theo nhận thức u, MAX theo thực tế v):**
  - **Nút Casper 1 (nhánh trái T2):** So sánh hai con là lá (2,2) và nút MAX (4,6). Pacman nghĩ Casper chọn min(u) = min(2, 4) = 2. Casper thực tế chọn max(v) = max(2, 6) = 6 → Cặp (u, v) = (2, 6).
  - **Nút Casper 2 (nhánh giữa T2):** So sánh hai con là lá (1,1) và nút MAX (3,5). Pacman nghĩ Casper chọn min(u) = min(1, 3) = 1. Casper thực tế chọn max(v) = max(1, 5) = 5 → Cặp (u, v) = (1, 5).
  - **Nút Casper 3 (nhánh phải T2):** So sánh hai con (3,9) và (4,8). Pacman nghĩ Casper chọn min(u) = min(3, 4) = 3. Casper thực tế chọn max(v) = max(9, 8) = 9 → Cặp (u, v) = (3, 9).
- **Tầng 1 (Tầng Pacman - MAX):**
  - **Nút Pacman trái (T1):** Pacman chọn giữa hai con Casper (2,6) và (1,5). Pacman chọn max(u) = max(2, 1) = 2 → Chọn nhánh (2,6) → Cặp (u, v) = (2, 6).
  - **Nút Pacman phải (T1):** Pacman chọn giữa hai con (3,9) và (4,8). Pacman chọn max(u) = max(3, 4) = 4 → Chọn nhánh (4,8) → Cặp (u, v) = (4, 8).
- **Tầng 0 (Nút gốc - Casper):**
  - So sánh hai nhánh Pacman (2,6) và (4,8). Pacman nghĩ Casper chọn min(u) = min(2, 4) = 2. Casper thực tế chọn max(v) = max(6, 8) = 8 → Cặp (u, v) tại gốc = (2, 8).
- **Kết luận: Giá trị thực tế thu được của trò chơi là 8.**

**b) Theo thứ tự duyệt trái sang phải, đánh dấu các nhánh có thể không cần duyệt khi Casper tính giá trị trò chơi.**

**Các nhánh bị Pacman bỏ qua/cắt bỏ do nhận thức sai [X]:** Nhánh (1,7), Nhánh (0,6), Nhánh (1,5). Casper biết Pacman chọn nước đi dựa trên u. Nếu ở một nhánh u chắc chắn không thể vượt qua giá trị u tốt nhất mà Pacman sẽ chọn ở các nhánh khác, Casper không cần tính chi tiết v của nhánh đó (cắt tỉa dựa trên u).

**c) Nếu Pacman biết Casper thân thiện thì giá trị toàn bộ cây thay đổi như thế nào?**

Khi Pacman biết Casper thân thiện, cả hai người chơi cùng phối hợp để tối đa hóa tiện ích (trò chơi hợp tác hoàn toàn MAX-MAX). Khi đó, Pacman và Casper sẽ luôn chọn nhánh dẫn tới lá có giá trị lớn nhất toàn cây. Giá trị trò chơi lúc này đạt cực đại là 9.

## Bài 6. Đánh giá thuật toán Tìm kiếm và Cập nhật chi phí A*

Với mỗi thuật toán sau, xác định điều kiện để được đảm bảo đầy đủ (Complete) và tối ưu (Optimal). Phân biệt tìm kiếm trên cây và trên đồ thị nếu cần.

| Thuật toán | Điều kiện đầy đủ | Điều kiện tối ưu |
|---|---|---|
| Breadth-First Search (BFS) | Đầy đủ nếu hệ số phân nhánh b hữu hạn. | Tối ưu nếu chi phí mỗi bước bằng nhau (đáp ứng bằng nhau). |
| Depth-First Search (DFS) | Đầy đủ trên không gian trạng thái hữu hạn (hoặc Graph search có cơ chế tránh lặp/chu trình). | Không tối ưu (có thể trả về giải pháp ở độ sâu lớn hơn giải pháp tối ưu). |
| Iterative Deepening Search (IDS) | Đầy đủ nếu hệ số phân nhánh b hữu hạn. | Tối ưu nếu chi phí mỗi bước bằng nhau. |
| A* Search | Đầy đủ nếu tồn tại giải pháp và chi phí mỗi bước $> \varepsilon > 0$. | Tree search: h(n) Admissible (chấp nhận được). Graph search: h(n) Consistent (nhất quán). |

**Giải thích vì sao điều kiện chi phí bước và đặc tính heuristic có thể ảnh hưởng đến kết luận.**

Chi phí bước bằng 0 có thể gây ra vòng lặp vô tận (mất tính đầy đủ) nếu không gian trạng thái chứa chu trình (cycles) và thuật toán không sử dụng cơ chế kiểm tra/loại bỏ trạng thái đã thăm (visited set / cycle detection). Nếu chi phí không đồng nhất, BFS/IDS không đảm bảo tối ưu. Heuristic không chấp nhận được (overestimating) làm A* bỏ qua đường đi tối ưu thực sự.

## Bài 7. A* khi chi phí một cạnh thay đổi

Sau khi A* graph search tìm được đường đi tối ưu từ trạng thái đầu đến đích, chi phí cạnh $X \rightarrow Y$ thay đổi từ c thành $c + n$ hoặc $c - n$, với $n > 0$. Với từng tình huống, hãy phân tích liệu chi phí tối ưu có thay đổi và cách cập nhật/tìm đường tối ưu mới thay vì nhất thiết chạy lại từ đầu. Khi giảm chi phí, giả sử chi phí mới vẫn hợp lệ theo mô hình tìm kiếm.

**a) Chi phí tăng n; cạnh $X \rightarrow Y$ thuộc đường tối ưu cũ và đã được khám phá.**

Chi phí tối ưu mới C'* thỏa mãn $C^* \leq C'^* \leq C^* + n$ (không nhất thiết tăng đúng n vì giải thuật có thể chuyển sang một đường đi thay thế khác có chi phí nhỏ hơn $C^* + n$). Cập nhật $g(Y) = g(Y) + n$ và các đỉnh con trong cây tìm kiếm, đưa Y vào lại Priority Queue (Open list) để tiếp tục giải thuật A* từ trạng thái hiện tại.

**b) Chi phí giảm n; cạnh $X \rightarrow Y$ thuộc đường tối ưu cũ và đã được khám phá.**

Đường đi tối ưu cũ vẫn là đường đi tối ưu, chi phí tối ưu giảm đúng n. Chỉ cần cập nhật giảm chi phí tích lũy g cho Y và các đỉnh hậu duệ.

**c) Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.**

Chi phí và đường đi tối ưu cũ không thay đổi (vì X → Y vốn dĩ đã không thuộc đường tối ưu).

**d) Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.**

Chi phí tối ưu mới qua X → Y có thể nhỏ hơn đường đi cũ. Cập nhật g(Y) = min(g(Y), g(X) + c - n). Nếu g(Y) giảm, thêm Y vào Open list để lan truyền cập nhật.

**e) Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá.**
**Lời giải:**

Chi phí và đường đi tối ưu không thay đổi.

**f) Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá.**
**Lời giải:**

Nếu g(X) + (c - n) + h(Y) < g(Goal), cạnh này có tiềm năng cho đường đi tốt hơn. Cần thêm Y vào Open list và tiếp tục chạy A*.

## Bài 8. Tác tử thông minh và môi trường

Xét ba kiểu tác tử, trong đó các hàm f_A, f_B, f_C có thể ngẫu nhiên nhưng không lưu trạng thái nội bộ:

- **A:** act(percept) → f_A()
- **B:** act(percept) → f_B(percept)
- **C:** lưu danh sách percepts; mỗi lần act(percept) thêm percept rồi trả về f_C(percepts)

Với mỗi đặc tính môi trường sau, hãy liệt kê các lớp tác tử có thể tồn tại một hàm f để hành động hợp lý hoàn hảo trong mọi môi trường thuộc lớp đó. Giải thích ngắn gọn.

**a) Quan sát đầy đủ; xác định; rời rạc; một tác tử; tĩnh.**

- **Các lớp tác tử:** B, C.
- **Giải thích:** Môi trường quan sát đầy đủ và tĩnh cho phép tác tử phản xạ đơn giản (Reflex agent B) và tác tử lưu lịch sử (C) hành động hợp lý hoàn hảo dựa trên percept hiện tại hoặc chuỗi percept. Loại A vì tác tử A chọn hành động độc lập với percept nhận được, do đó không thể đảm bảo hành động hợp lý trong mọi môi trường thuộc lớp này.

**b) Quan sát một phần; xác định; rời rạc; một tác tử; tĩnh.**

- **Các lớp tác tử:** C.
- **Giải thích:** Quan sát một phần đòi hỏi tác tử phải lưu trữ chuỗi lịch sử nhận thức (percept sequence) để suy luận ra trạng thái ẩn của môi trường. Chỉ có tác tử C duy trì danh sách percepts mới thực hiện được.

**c) Quan sát một phần; ngẫu nhiên; rời rạc; một tác tử; tĩnh.**

- **Các lớp tác tử:** C.

- **Giải thích:** Môi trường ngẫu nhiên và quan sát một phần cần toàn bộ thông tin lịch sử quan sát để tính toán phân phối xác suất trạng thái (belief state) nhằm tối đa hóa tiện ích kỳ vọng.

**d) Đúng hay sai: Có tồn tại một môi trường mà mọi tác tử đều hành động hợp lý. Nêu ví dụ hoặc phản biện.**

**Kết luận:** ĐÚNG.

**Ví dụ:** Môi trường có tập hành động chỉ gồm đúng 1 hành động duy nhất, hoặc môi trường mà mọi hành động đều mang lại độ đo hiệu quả (performance measure) như nhau. Khi đó bất kỳ tác tử nào thực hiện hành động gì cũng đều được xem là hành động hợp lý.